

数据库概论

在线练习

2021秋线上授课，教学直播、讨论所使用的平台是：

- ☐ A 雨课堂
- ☒ B 腾讯会议
- ☐ C QQ直播间
- ☐ D 钉钉

提交

2021秋线上授课，全程授课录像等教学资源，使用的是：

- ☐ A 中国大学MOOC
- ☐ B 雨课堂
- ☐ C 超星
- ☒ D 爱课程网上的精品资源共享课

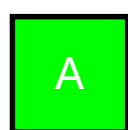
提交

2021秋线上授课，课堂考勤、互动练习答题、课堂数据记录，使用的是：

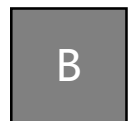
- ☐ A QQ
- ☐ B 微信
- ☒ C 雨课堂
- ☐ D 钉钉

提交

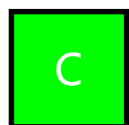
Files vs. Databases



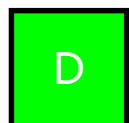
文件本身没有数据模型，
就是字符流



文件不能用来管理数据



数据库支持语义丰富的数据
模型，可更有效地管理数据



文件和数据库都可以管理
数据

提交

关于数据、数据模型、数据模式

A

数据是信息的表现形式

B

数据模型是表达现实世界信息的一组数据结构和规则

C

数据模式是某个具体数据库的数据结构的描述

D

数据模型和数据模式是一回事

提交

关于数据库数据的物理独立性和逻辑独立性

A

数据独立性是依靠数据库的三级数据模式和它们之间的两级映射实现的

B

物理独立性是指当数据库的逻辑结构改变时，上层应用不受影响

C

逻辑独立性是指当数据库的物理存储结构改变时，上层应用不受影响

D

物理独立性是指当数据库的物理存储结构改变时，上层应用不受影响

提交

关于数据模型的发展

- ☐ A 关系模型和网状模型同时出现
- ☐ B 层次模型和关系模型同时出现
- ☒ C 先有层次模型后有关系模型
- ☐ D 层次模型和网状模型是一回事

提交

关于基于数据库的应用系统体系结构

A

早期的应用系统都是集中式结构

B

分布式结构和Client/Server都是由多台计算机联网，所以它们是一回事

C

站在数据库管理系统的角度，C/S结构和B/S结构并无本质差别

D

移动计算结构其实是一种基于移动通讯网络的C/S或者B/S结构

提交

OLTP是数据库技术最早的应用领域，关于OLTP正确的说法是：

A

OLTP指On-Line Transaction Processing，联机事务处理

B

OLTP就是在线查询

C

OLTP应用是指基于结构化数据的日常事务性处理，比如银行存取款业务

D

OLTP是关系数据模型最擅长的应用领域

提交

关于Data Warehouse, OLAP, Data Mining的含义, 下面正确的是:

- ☐ A 它们是指仓库管理、物流
- ☐ B 是基于数据库的日常事务性管理
- ☐ C 是指挖矿、采掘工程
- ☒ D 它们是数据库技术新的应用领域, 支持基于数据的辅助决策分析

提交

一个数据库系统是由数据库、DBMS、DBA、应用程序、最终用户组成的生态，下面说法正确的是：

A

DBMS是指DataBase Management System，数据库管理系统

B

DBA是指DataBase Administrator，数据库管理员

C

在实现了信息化管理的企业里，DBA是一个非常重要的职位

D

最终用户可以使用数据库系统中的任何功能，访问任何资源

提交

关于数据库系统的生命周期：

A

数据库设计是指设计DBMS

B

数据库设计是指设计能够满足某个应用系统需求的数据库的结构（Schema）

C

数据库设计时不需要考虑服务器硬件、操作系统的特性

D

数据库设计时需要考虑所用DBMS的特性

提交

关于层次数据模型，正确的说法是：

- ☒ A 它是数据库技术早期采用的一种数据模型
- ☐ B 层次模型的基本数据结构是链表
- ☒ C 层次数据模型的基本数据结构是树
- ☐ D 层次数据模型无法表达现实世界中的多对多联系

提交

关于网状数据模型，正确的说法是：

- ☒ A 它是数据库技术早期采用的一种数据模型
- ☒ B 网状模型的基本数据结构是链表
- ☐ C 网状数据模型的基本数据结构是树
- ☐ D 网状数据模型无法表达现实世界中的多对多联系

提交

关于层次数据模型和网状数据模型对现实世界中多对多联系的表达，下面正确的是：

A

它们采用的方法是一样的

B

层次模型通过引入虚记录来表达记录型之间的多对多联系

C

网状模型通过引入LINK记录来表达记录型之间的多对多联系

D

虚记录和LINK记录的实现都涉及复杂的指针操作，对用户不友好

提交

关于关系数据模型，下面说法正确的是：

- ☐ A 关系模型的基本数据结构是堆文件
- ☒ B 关系模型的基本数据结构是表
- ☐ C 关系模型的基本数据结构是数组
- ☐ D 关系模型的基本数据结构是链表

提交

对于关系模型与层次模型、网状模型之间的区别，下述说法错误的是：

- ☐ A 关系模型统一用表来表达现实世界中的实体集及其联系
- ☐ B 层次模型和网状模型都涉及了指针等底层编程细节
- ☒ C 关系模型的概念比层次、网状数据模型复杂
- ☐ D 关系模型抽象层次更高，屏蔽了数据结构、指针等底层细节

提交

我们说关系模型通过一种“软连接”（Soft Link）实现了现实世界实体之间联系的表达，下面说法正确的是：

- ☐ A 软连接就是指针
- ☒ B 关系模型通过比较不同表的公共属性值是否相等，来判断记录之间是否有联系，而不是通过物理指针
- ☐ C 软连接因为指针是软的
- ☐ D 依据软连接实现查询的效率高于物理指针

提交

对于关系模型中的第一范式（1NF）和空值（NULL），下面说法正确的是：

A

1NF是指关系属性的类型只能是整型和字符串型

B

NULL是指关系中某条元组的某个属性的值不知道

C

1NF是指关系中每个属性的类型只能是基本数据类型

D

NULL是指关系中某条元组的某个属性的值为空串

提交

关于一个关系的主键，下面说法正确的是：

A

一个关系可以有多个候选键

B

一个关系可以有多个主键

C

一个关系只能有一个主键

D

主键可以为空值

提交

关于外键和引用完整性，下面说法不正确的是：

- ☐ A 外键可以理解为逻辑指针
- ☐ B 外键可以为空值
- ☒ C 一个关系只能有一个外键
- ☐ D 外键的值在它所引用的表的相应属性中必须存在

提交

$\{\sigma, \pi, \cup, -, \times\}$ 构成了关系代数完备的操作集，意思是说：

- ☐ A 这5个操作已经完备了，系统中不要再支持其它操作
- ☒ B 对关系的任何操作都可以由这5个操作推导出来
- ☒ C 一个关系数据库系统只要支持这5个操作，理论上它的功能就是完整的
- ☒ D 一个实际的数据库系统可以根据需要支持其它操作

提交

按照投影操作的定义，为什么要消除结果中可能出现的重复元组？

而在一个实际的数据库系统中，为什么它一般不会主动地消除结果中的重复元组，除非用户要求它这么做？

正常使用主观题需2.0以上版本雨课堂

作答

为什么笛卡尔乘积在实际应用中直接的用途不多？但是它又很重要？

A

虽然它能够将两个表拼接起来，但是会产生大量无意义的元组组合

B

实际应用中不需要

C

它会产生错误结果

D

它是实现连接操作的基础

提交

关于自然连接，下述说法正确的是：

- ☒ A 它是在笛卡尔乘积的基础上，做公共属性值相等的选择
- ☒ B 结果Schema中属性个数少于笛卡尔乘积
- ☐ C 结果的Schema和笛卡尔乘积完全相同
- ☒ D 结果的元组数一般会少于笛卡尔乘积

提交

除法操作可以用关系代数基本操作推导出来，对于关系 $A(X,Y)$ 和 $B(Y)$ ，则 A/B 等于：

- ☐ A $\pi_X ((\pi_X(A) \times B) - A)$
- ☐ B $\pi_X(A)$
- ☐ C $\pi_X ((\pi_X(A) \times B) - A) - \pi_X(A)$
- ☒ D $\pi_X(A) - \pi_X ((\pi_X(A) \times B) - A)$

提交

关于外并操作，下面说法正确的是：

- ☒ A 它可以将两个不满足并兼容条件的表做集合并
- ☒ B 结果的元组数是参与运算的两个表元组数的和
- ☐ C 结果的Schema和笛卡尔乘积是一样的
- ☐ D 结果的元组数是参与运算的两个表元组数的积

提交

关于关系演算和关系代数之间的关系，下述说法正确的是：

A

它们都是非过程化的表达方法

B

关系演算可能会出现不安全的查询

C

关系代数可能会出现不安全的查询

D

关系演算和关系代数的表达能力是等价的

提交

关于域关系演算和元组关系演算，下面说法正确的是：

A

域关系演算的变量可以定义在元组上

B

它们是非过程化的关系运算表达方法

C

域关系演算的变量定义在属性上

D

元组关系演算的变量定义在元组上

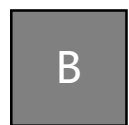
提交

假设 $Sailors = (D, N, T, A)$, 对于查询 $\{ t[N] \mid t \in Sailors \wedge t.T > 7 \wedge t.A < 50 \}$, 下述说法正确的是:



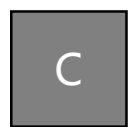
A

这是一个元组关系演算表达式



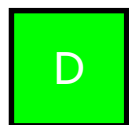
B

这是一个域关系演算表达式



C

它和关系代数表达式 $\pi_t \sigma_{T > 7 \wedge A < 50} (Sailors)$ 等价

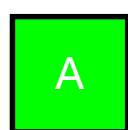


D

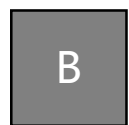
它和关系代数表达式 $\pi_N \sigma_{T > 7 \wedge A < 50} (Sailors)$ 等价

提交

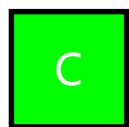
关于传统数据模型，下述说法正确的是：



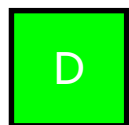
层次、网状、关系数据模型统称为传统数据模型



关系、ER、面向对象数据模型统称为传统数据模型



它们都适用于OLTP应用



它们都以记录为基础表达现实世界中的实体

提交

关于ER数据模型，下面说法正确的是：

A

它不能作为数据库系统的数据模型

B

作为副产品，ER图在数据库概念设计阶段很有用

C

可以实现基于ER模型的数据库管理系统

D

ER模型的表达能力比关系模型强

提交

关于面向对象数据模型，下面说法哪个是错误的：

- ☐ A 面向对象数据模型的表达能力比关系模型要强
- ☒ B 面向对象数据模型仅适用于程序设计语言，不适合数据库系统
- ☐ C 对象-关系型DBMS是在关系型内核的基础上做了一定扩展
- ☐ D 对象-关系型DBMS并不能支持真正的面向对象数据模型

提交

关于数据模型的发展，下述说法正确的是：

A

关系模型解决了已知的大部分问题，不需要再研究其它数据模型了

B

ER、面向对象、时态、XML等等数据模型都可称为“后关系”数据模型

C

关系模型仍然是主流数据模型

D

随着应用的发展，新的需求层出不穷，数据模型的研究是无止境的

提交

关于数据库访问接口所包含的形式，下面不正确的是：

- ☐ A 查询语言
- ☒ B 面向对象程序设计语言
- ☐ C GUI界面和维护工具
- ☐ D 支持访问的类库

提交

关于QBE语言，下述说法正确的是：

- ☒ A 它是Query By Example(通过例子进行查询) 的简称
- ☒ B 它是域关系演算语言的典型代表
- ☒ C 它是一种表格式查询语言(TQL)
- ☐ D 它是一种形式化查询语言

提交

关于查询语言（QL）不是“Turing complete”的，是指：

- ☐ A 像QBE这种表格式查询语言不具备编程能力
- ☐ B 图像化查询语言不具备编程能力
- ☐ C 形式化查询语言只具备简单的编程能力
- ☒ D 各种查询语言均不具备编程能力，只支持对数据库中海量数据的有效访问

提交

SQL语言的数学基础是：

- ☐ A 关系代数
- ☒ B 元组关系演算
- ☐ C 域关系演算
- ☐ D 集合论

提交

假设Demo表的定义如下：

```
create table demo
```

```
(
```

```
id int primary key ,
```

```
name char (10) ,
```

```
content int,
```

```
city char(10) default 'beijing'
```

```
)
```

则执行插入语句insert into demo

(id ,name ,content) values (2,'lisi',119) 之后,

lisi的content和city属性的值分别是：

A

119, Shanghai

B

110, Beijing

C

119, Beijing

D

119, Nanjing

提交

关于一条SQL语句的Conceptual Evaluation Strategy，我们说这是为了帮助理解概念，实际的系统一般都不会真的按此流程去做，这是指：

- ☐ A 这个处理逻辑是错误的
- ☐ B 这只是理论，无法实现
- ☐ C 这样做系统实现很困难
- ☒ D 这样做逻辑虽然正确，但执行效率比较低

提交

```
SELECT S.sid  
FROM Sailors S, Reserves R  
WHERE S.sid=R.sid;
```

查找预定过船的水手，关于这条语句下面正确的说法是：

A

如果只需查预定过船的水手编号，可以只用Reserves表

B

增加DISTINCT选项，对结果的语义没有影响

C

如果要查预定过船的水手姓名，只需将SELECT中的S.sid换成S.Sname

D

如果SELECT中的S.sid换成S.Sname，增加DISTINCT选项会影响结果的语义

提交

```
SELECT S.age  
FROM Sailors S  
WHERE S.sname LIKE 'B_%a';
```

对上述语句，以下水手的年龄将会出现在结果中：

A

Bob

B

Barbra

C

Bainbridge

D

Blendena

提交

```
SELECT S.sid
FROM Sailors S, Boats B, Reserves R
WHERE S.sid=R.sid AND R.bid=B.bid
      AND (B.color= 'red' OR B.color= 'green' );
```

如果将上述语句中的OR直接换成AND，结果将是：

- ☐ A 预定过红船和绿船的水手
- ☐ B 预定过红船的水手
- ☐ C 预定过红船或者绿船的水手
- ☒ D 结果为空

提交

```
SELECT S.sid  
FROM Sailors S, Boats B, Reserves R  
WHERE S.sid=R.sid AND R.bid=B.bid  
      AND B.color= 'red'  
  
UNION  
SELECT S.sid  
FROM Sailors S, Boats B, Reserves R  
WHERE S.sid=R.sid AND R.bid=B.bid  
      AND B.color= 'green' ;
```

如果将上述语句中的UNION换成INTERSECT，结果将是：

- ☒ A 预定过红船和绿船的水手
- ☐ B 预定过红船的水手
- ☐ C 预定过红船或者绿船的水手
- ☐ D 结果为空

提交

如何通过最少的改动，将下述查询改为查找预定过红船和绿船的水手的姓名？

```
SELECT S.sid
FROM Sailors S, Boats B, Reserves R
WHERE S.sid=R.sid AND R.bid=B.bid
      AND B.color= 'red'
INTERSECT
SELECT S.sid
FROM Sailors S, Boats B, Reserves R
WHERE S.sid=R.sid AND R.bid=B.bid
      AND B.color= 'green' ;
```

正常使用主观题需2.0以上版本雨课堂

作答

关于用关联嵌套子查询查找预定过103号船的水手，下述说法正确的是：

- ☒ A 对外层查询中Sailors表的每条元组，子查询都要执行一次，因此效率低
- ☐ B 对外层查询中Sailors表的每条元组，子查询只要执行一次，因此效率高
- ☐ C 关联嵌套和非关联嵌套的效率一样
- ☐ D 使用非关联嵌套的效率低

提交



适当修改下列语句，使之能够查找预定过103号船并且只预定过一次的水手：

```
SELECT S.sname
FROM Sailors S
WHERE EXISTS
    (SELECT *
     FROM Reserves R
     WHERE R.bid=103 AND S.sid=R.sid);
```

正常使用主观题需2.0以上版本雨课堂

作答


```
SELECT S.sname
FROM Sailors S
WHERE EXISTS (SELECT *
              FROM Reserves R1
              WHERE R1.bid=103 AND R1.sid=S.sid AND
                NOT EXIST (SELECT *
                          FROM Reserves R2
                          WHERE R2.bid=103 AND
                                R2.sid=R1.sid AND
                                R1.day  $\neq$  R2.day
                          )
            );
```

```
SELECT S.sname
FROM Sailors S
WHERE EXISTS (SELECT *
              FROM Reserves R1
              WHERE R1.bid=103 AND R1.sid=S.sid)
              AND NOT EXIST (SELECT *
                             FROM Reserves R2
                             WHERE R2.bid=103 AND
                                    R2.sid=R1.sid AND
                                    R1.day  $\neg$ = R2.day
                             );
```

查找的是：

- ☐ A 预定过103号船的水手
- ☐ B 预定过103号船并且只预定过一次的水手
- ☒ C 有逻辑错误
- ☐ D 没有预定过103号船的水手

提交

```
SELECT *  
FROM Sailors S  
WHERE S.rating > ALL (SELECT S2.rating  
                      FROM Sailors S2  
                      WHERE S2.sname='Horatio');
```

查找的是：

- ☐ A 级别比某个名叫Horatio的水手高的水手
- ☒ B 级别比所有名叫Horatio的水手高的水手
- ☐ C 级别和名叫Horatio的水手一样的水手
- ☐ D 级别比所有名叫Horatio的水手低的水手

提交

```
SELECT S.sid
FROM Sailors S, Boats B, Reserves R
WHERE S.sid=R.sid AND R.bid=B.bid AND B.color='red'
      AND S.sid NOT IN (SELECT S2.sid
                        FROM Sailors S2, Boats B2, Reserves R2
                        WHERE S2.sid=R2.sid AND R2.bid=B2.bid
                        AND B2.color='green')
```

查找的是：

- ☐ A 预定过红船和绿船的水手
- ☐ B 预定过红船或者绿船的水手
- ☐ C 只预定过红船的水手
- ☒ D 预定过红船但没预定过绿船的水手

提交



修改下列语句，查找预定过所有红船的水手：

```
SELECT S.sname
FROM Sailors S
WHERE NOT EXISTS
    (SELECT B.bid
     FROM Boats B
     WHERE NOT EXISTS
        (SELECT R.bid
         FROM Reserves R
         WHERE R.bid=B.bid AND R.sid=S.sid))
```

正常使用主观题需2.0以上版本雨课堂

作答

为什么下列语句是非法的：
SELECT S.sname, MAX (S.age)
FROM Sailors S

- ☐ A 因为有语法错误
- ☐ B SELECT中不能用MAX
- ☒ C 水手有很多，而最大年龄只有一个，无法确定对应关系，逻辑有矛盾
- ☐ D 因为水手会有重名

提交

在使用聚集函数时，从语法上要求出现在SELECT子句的属性必须也出现在GROUP BY子句中，相关原因正确的是：

A

SELECT和GROUPBY是配对的

B

每个分组产生一条结果元组，因此SELECT子句中的属性必须在每个组具有单一值

C

通过语法的约束解决复杂的单一值语义问题

D

数据库系统无法从业务语义上判断SELECT子句中的属性是否对每个分组具有单一值

提交

为什么HAVING子句也从语法上要求出现在HAVING子句的属性必须也出现在GROUP BY子句中？

- ☐ A 因为HAVING必须和GROUP BY成对出现
- ☐ B 为了使用简单
- ☒ C HAVING是组筛选条件，因此其属性必须在每个组具有单一值
- ☐ D 因为HAVING和SELECT地位平等

提交

SELECT S.rating, MIN (S.age) AS minage
FROM Sailors S
WHERE S.age >= 35
GROUP BY S.rating
HAVING COUNT (*) > 1
的正确结果是：

- ☐ A 3, 63.5
- ☒ B 7, 35.0
- ☐ C 9, 35.0
- ☐ D 10, 35.0

<u>sid</u>	sname	rating	age
22	dustin	7	45.0
29	brutus	1	33.0
31	lubber	8	55.5
32	andy	8	25.5
58	rusty	10	35.0
64	horatio	7	35.0
71	zorba	10	16.0
74	horatio	9	35.0
85	art	3	25.5
95	bob	3	63.5
96	frodo	3	25.5

提交

```
SELECT B.bid, COUNT (*) AS scout  
FROM Boats B, Reserves R  
WHERE R.bid=B.bid AND B.color='red' AND  
       R.day>'01/07/2019'  
GROUP BY B.bid  
查找的是：
```

- ☐ A 每条红船的预定次数
- ☐ B 每条绿船的预定次数
- ☒ C 每条红船2019.7.1之后的预定次数
- ☐ D 每条红船的平均预定次数

提交

```
SELECT S.rating, MIN (S.age)
FROM Sailors S
WHERE S.age > 18
GROUP BY S.rating
HAVING 2 < (SELECT COUNT (*)
            FROM Sailors S2
            WHERE S2.rating = S.rating)
```

的正确结果是：

- ☒ A 3, 25.5
- ☐ B 7, 35.0
- ☐ C 8, 25.5
- ☐ D 10, 35.0

<u>sid</u>	sname	rating	age
22	dustin	7	45.0
29	brutus	1	33.0
31	lubber	8	55.5
32	andy	8	25.5
58	rusty	10	35.0
64	horatio	7	35.0
71	zorba	10	16.0
74	horatio	9	35.0
85	art	3	25.5
95	bob	3	63.5
96	frodo	3	25.5

提交

```
SELECT S.rating  
FROM Sailors S  
WHERE S.age = (SELECT MIN (AVG (S2.age))  
               FROM Sailors S2)
```

上述语句试图查询平均年龄最小的级别，但它是错误的，其错误在于：

A

聚集函数不能嵌套使用

B

一条语句不能用两个聚集函数

C

即使能嵌套使用聚集函数，句子的逻辑也不对

D

应该先算最小值再算平均值

提交

空值带来了一些新的问题，如果 $\text{age}=22$ ， sex 为空值，则布尔表达式 $\text{age} > 20 \text{ AND } \text{sex} = 'm'$ 的结果为：

- ☐ A True
- ☒ B null (unknown)
- ☐ C False
- ☐ D 随机

提交

```
SELECT type, sum(accidents)/sum(hours_used)
FROM Machines
GROUP BY type;
```

对上述语句，下面说法正确的是：

- ☐ A 可以正常执行，没问题
- ☒ B 有可能出现表达式被0除
- ☐ C 不能使用多个聚集函数
- ☐ D 出现聚集函数的嵌套使用

提交

```
SELECT type, sum(accidents)/sum(hours_used)
FROM Machines
GROUP BY type;
```

如果认为这条语句有缺陷，应如何改正它？

正常使用主观题需2.0以上版本雨课堂

作答

```
SELECT d.deptname  
FROM dept AS d  
WHERE (SELECT avg(bonus)  
        FROM emp  
        WHERE deptno=d.deptno)  
       > (SELECT salary  
          FROM emp  
          WHERE empno='09018000');
```

设empno为为雇员编号，则该语句查询的是：

- ☐ A 平均奖金高于09018000号雇员奖金的部门名称
- ☐ B 平均薪水高于09018000号雇员奖金的部门名称
- ☐ C 平均薪水高于09018000号雇员薪水的部门名称
- ☒ D 平均奖金高于09018000号雇员薪水的部门名称

提交


```
SELECT d.deptno, d.deptname, (SELECT MAX (salary)
                             FROM emp
                             WHERE deptno=d.deptno) AS maxpay
FROM dept AS d
WHERE d.location = 'Nanjing' ;
```

```
SELECT d.deptno, d.deptname, MAX(e.salary) AS maxpay
FROM dept AS d, emp AS e
WHERE d.deptno=e.deptno AND d.location = 'Nanjing'
GROUP BY d.deptno, d.deptname;
```

对于上述两条查询语句的结果，以下正确的是：

- ☐ A 完全不是一回事
- ☒ B 结果完全相同
- ☐ C 结果略有不同
- ☐ D 第一条查的是南京部门的员工的最高工资，而第二条不是

提交

```
SELECT reyear, max(pay)
FROM (SELECT name, salay+bonus AS pay,
            year(redate) AS reyear
      FROM retiredemp) AS emp2
GROUP BY reyear;
```

设redate为员工退休日期，则该语句查询的是：

- ☐ A 退休员工的最高总收入
- ☐ B 每个年份的退休员工的平均总收入
- ☒ C 每个年份的退休员工的最高总收入
- ☐ D 每个年份的退休员工的平均总收最高薪水

提交

```
WITH payroll (deptno, totalpay) AS  
  (SELECT deptno, sum(salary)+sum(bonus)  
   FROM emp  
   GROUP BY deptno)  
SELECT deptno  
FROM payroll  
WHERE totalpay >=(SELECT avg(totalpay)  
                  FROM payroll);
```

该语句查询的是：

- ☐ A 总收入最高的部门
- ☒ B 总收入在平均总收入之上的部门
- ☐ C 总收入等于平均总收入的部门
- ☐ D 总收入低于平均总收入的部门

提交

在PPT计算Teacher和Course外连接操作的例子中，如果将其中的EXCEPT ALL和UNION ALL换成EXCEPT和UNION，会怎么样？

- ☒ A 不影响结果，但使用EXCEPT、UNION性能会降低
- ☐ B 不影响结果，但使用EXCEPT、UNION性能可提高
- ☐ C 会影响结果，多出重复元组
- ☐ D 会影响结果，损失有效元组

提交

```
WITH agents (name, salary) AS
  ((SELECT name, salary
    FROM FedEmp
    WHERE manager='Hoover')
  UNION ALL
  (SELECT f.name, f.salary
    FROM agents AS a, FedEmp AS f
    WHERE f.manager = a.name))
SELECT name
FROM agents
WHERE salary = (SELECT MAX(salary)
  FROM agents);
```

该语句查询的是：

- ☐ A Hoover的所有属下中平均薪水最高的雇员姓名
- ☐ B Hoover的直接属下中薪水最高的雇员姓名
- ☒ C Hoover的所有属下中薪水最高的雇员姓名
- ☐ D Hoover的最高薪水

提交

WITH wingpart (part, subpart, qty) AS
((SELECT part, subpart, qty
FROM components
WHERE part= 'wing')
UNION ALL
(SELECT w.subpart, c.subpart, w.qty*c.qty
FROM wingpart w, components c
WHERE w.subpart=c.part))
SELECT sum(qty) AS qty
FROM wingpart
WHERE part='hinge' AND subpart= 'rivet' ;
这条语句的执行结果是：

A 20

B 50

C 5

D 16

wingpart		
Subpart	QTY	
strut	5	Used directly
aileron	1	Used directly
landing gear	1	Used directly
rivet	100	Used directly
rivet	50	Used on strut
hinge	2	Used on aileron
rivet	5	Used on aileron
hinge	3	on landing gear
rivet	8	on landing gear
rivet	8	on aileron hinges
rivet	12	on L G hinges

提交

如果把航班路径递归搜索例子中的下面两个终止条件去掉，结果会怎么样？

AND f.destination<>'SFO'

AND f.origin<>'JFK'

- ☐ A 会无限递归，无法终止
- ☐ B 递归可以终止，但结果错误
- ☐ C 会遗漏正确的路径
- ☒ D 仍然可以获得正确结果，但会产生一些无意义的路径，影响效率

提交

在航班路径搜索的例子中，通过递归查询计算trips临时表的过程，属于下述哪种算法思想？

☒ A 广度优先搜索

☐ B 深度优先搜索

☐ C 随机搜索

☐ D 回溯算法

提交

UPDATE Sailors SET age = 36 WHERE sid = '71';
 SELECT S.rating, MIN (S.age) AS minage
 FROM Sailors S
 WHERE S.age > 35
 GROUP BY S.rating
 HAVING COUNT (*) > 1;
 的正确结果是:

- ☐ A 7, 45
- ☐ B 3, 25.5
- ☒ C 10, 36
- ☐ D 8, 55.5

<u>sid</u>	sname	rating	age
22	dustin	7	45.0
29	brutus	1	33.0
31	lubber	8	55.5
32	andy	8	25.5
58	rusty	10	45.0
64	horatio	7	35.0
71	zorba	10	16.0
74	horatio	9	35.0
85	art	3	25.5
95	bob	3	63.5
96	frodo	3	25.5

提交

关于视图（VIEW）和用WITH定义的公共表表达式，下述说法正确的是：

A

它们都是虚表

B

它们都不能被更新

C

视图在一定条件下可以更新

D

视图的定义永久存在，除非被删除

提交

```
CREATE VIEW Ratingavg AS  
    SELECT rating, AVG(age)  
    FROM Sailors  
    GROUP BY rating;
```

对于该视图，下述说法正确的是：

- ☐ A 它可以被更新
- ☒ B 它不能被更新
- ☐ C 去掉GROUP BY就能更新
- ☐ D 增加HAVING就能更新

提交

程序设计语言和数据库系统之间存在所谓的“阻抗不匹配”问题，关于此问题下属说法正确的是：

- ☒ A 它和程序设计语言与DBMS之间的数据交换有关
- ☒ B 数据库的处理对象是集合，而程序设计语言是变量
- ☐ C DBMS服务器和客户端PC的网卡的阻抗应该适配
- ☒ D 一个DBMS所支持的数据类型与程序设计语言支持的不完全一致

提交

解决程序设计语言和DBMS交互问题的解决方案一般有：

A

嵌入式SQL

B

破解DBMS，直接调用其API

C

由DBMS提供编程API，比如ODBC、JDBC等

D

将访问DBMS的API封装成类库，以便在OOP中使用数据库

提交

关于宿主变量，下述说法正确的是：

- ☒ A 它是程序设计语言和数据库之间交换数据的媒介
- ☒ B 它在程序设计语言中可以当做普通变量使用
- ☒ C 它可以出现在SQL语句中
- ☒ D 它需要专门定义

提交

关于SQLCA (SQL Communication Area), 下述说法正确的是:

A

这是一个用户可定制的结构

B

它负责用户进程和DBMS核心之间的通信

C

它是由系统定义的一个特殊的宿主变量

D

它负责向应用程序传递一条SQL语句执行结果状态的信息

提交

关于指示符变量，下属说法正确的是：

A

它是用来帮助计算属性值精度的

B

它是一种短整型的特殊宿主变量

C

它帮助我们在程序设计语言中处理空值

D

它是用来标示属性存放位置的

提交

假设宿主变量e_number, e_name, e_phone的值分别是“09018000”，“张三”，“52090861”，指示变量ind_phone的值为-1，则执行下述语句后，数据库中新插入的雇员信息为：

```
EXEC SQL INSERT INTO Employee VALUES  
(:e_number, :e_name, :e_phone:ind_phone );
```

A

<“09018000”，“张三”，“52090861”>

B

<“09018000”，“张三”，-1>

C

<“09018000”，“张三”，NULL>

D

<“09018000”，“张三”，“52090861-1”>

提交

```

:
EXEC SQL DECLARE C1 CURSOR FOR
        SELECT SNO, GRADE
        FROM SC
        WHERE CNO = :GIVENCNO;
EXEC SQL OPEN C1;
if (SQLCA.SQLCODE<0) exit(1); /* There is error in query*/
while (1) {
        EXEC SQL FETCH C1 INTO :SNO, :GRADE :GRADEI
        if (SQLCA.SQLCODE==100) break;
        /* treat data fetched from cursor, omitted*/
        :
}
EXEC SQL CLOSE C1;
:

```

如果学生09018000没有参加该门课考试（成绩为NULL），则循环处理到该学生时，宿主变量SNO, GRADE, GRADEI的值为：

- ☐ A "09018000" , 80, -1
- ☒ B "09018000" , 任意, -1
- ☐ C "09018000" , 85, 0
- ☐ D "09018000" , 任意, 0

提交

```
      :  
EXEC SQL DECLARE C1 CURSOR FOR  
      SELECT SNO, GRADE  
      FROM SC  
      WHERE CNO = :GIVENCNO;  
EXEC SQL OPEN C1;  
if (SQLCA.SQLCODE<0) exit(1); /* There is error in query*/  
while (1) {  
      EXEC SQL FETCH C1 INTO :SNO, :GRADE  
      if (SQLCA.SQLCODE==100) break;  
      /* treat data fetched from cursor, omitted*/  
      :  
}  
EXEC SQL CLOSE C1;  
      :
```

如果学生09018000没有参加该门课考试（成绩为NULL），则循环处理到该学生时，宿主变量SNO, GRADE的值为：

- ☐ A "09018000" , 80
- ☐ B "09018000" , 任意
- ☐ C "09018000" , -1
- ☒ D 出错(SQLCODE_NO_INDICATOR)

提交

在对嵌入了SQL的程序进行编译时，连接阶段需要连接SQL functions library库函数，这些库函数：

- ☒ A 是由数据库厂家提供的，类似于ODBC
- ☐ B 它就是ODBC
- ☐ C 它是JDBC
- ☐ D 它应该由应用开发人员提供

提交

关于动态SQL，下述说法正确的是：

A

它仍然属于嵌入式SQL

B

需要执行的SQL语句在编译时无法确定

C

需要执行的SQL语句需根据程序运行的状态和输入而定

D

需要执行的SQL语句随数据库的插删改操作同步变化

提交

```
        :  
EXEC SQL BEGIN DECLARE SECTION;  
char sqlstring[200];  
EXEC SQL END DECLARE SECTION;  
char cond[150];  
strcpy( sqlstring, " DELETE FROM STUDENT  
WHERE " );  
printf( " Enter search condition : " );  
scanf( "%s" , cond);  
strcat( sqlstring, cond);  
EXEC SQL EXECUTE IMMEDIATE :sqlstring;  
        :
```

- ☐ A 这是一个需要处理查询结果的动态SQL语句
- ☐ B 这是一个需要使用动态游标的动态SQL语句
- ☐ C 这是手动输入的动态SQL语句
- ☒ D 这是一个可直接执行的动态SQL语句

提交

```

:
EXEC SQL BEGIN DECLARE SECTION;
char sqlstring[200];
int birth_year;
EXEC SQL END DECLARE SECTION;
strcpy( sqlstring, "DELETE FROM STUDENT WHERE
        YEAR(BDATE) <= :y; ");
printf(" Enter birth year for delete :");
scanf("%d", &birth_year);
EXEC SQL PREPARE purge FROM :sqlstring;
EXEC SQL EXECUTE purge USING :birth_year;
:
    
```

- ☐ A 这是一个需要处理查询结果的动态SQL语句
- ☒ B 这是一个带动态参数的动态SQL语句
- ☐ C 这是手动输入的动态SQL语句
- ☐ D 这是一个可直接执行的动态SQL语句

提交

```

:
EXEC SQL BEGIN DECLARE SECTION;
char sqlstring[200];
int birth_year;
EXEC SQL END DECLARE SECTION;
strcpy( sqlstring, "DELETE FROM STUDENT WHERE
        YEAR(BDATE) <= :y");
printf(" Enter birth year for delete :");
scanf("%d", &birth_year);
EXEC SQL PREPARE purge FROM :sqlstring;
EXEC SQL EXECUTE purge USING :birth_year;
:
    
```

上述语句中*purge*的含义是：

- ☒ A 是程序员为这条动态SQL语句所起的名称标识
- ☐ B 是删除命令
- ☐ C 是宿主变量
- ☐ D 是STUDENT表的属性

提交


```

:
strcpy( sqlstring, " SELECT SNO,GRADE FROM SC WHERE CNO= :c " );
printf( " Enter the course number : " );
scanf( "%s" , GIVENCNO);
EXEC SQL PREPARE query FROM :sqlstring;
EXEC SQL DECLARE grade_cursor CURSOR FOR query;
EXEC SQL OPEN grade_cursor USING :GIVENCNO;
if (SQLCA.SQLCODE<0) exit(1);
while (1) {
    EXEC SQL FETCH grade_cursor INTO :SNO, :GRADE :GRADEI
    if (SQLCA.SQLCODE==100) break;
/* treat data fetched from cursor, omitted*/
:
}
EXEC SQL CLOSE grade_cursor;
:

```

对该程序片段，下述说法正确的是：

☒ A query是程序员为该动态SQL语句起的名称标识

☐ B query是游标的名称

☒ C grade_cursor是游标的名称

☒ D USING :GIVENCNO用类似宏替换的方法处理动态参数

提交

关于存储过程，下述说法正确的是：

A

它是一段编译好的、优化过的放在数据库服务器中的SQL语句脚本代码

B

它在系统初始化时一次性加载，可以多次被调用

C

因为比较复杂，所以它的执行效率低于普通SQL语句

D

因为事先编译优化好，所以它的执行效率高于普通SQL语句

提交

关于PPT中的存储过程例子
drop_student, 下述说法正
确的是:

- ☒ A 保证SC和STUDENT表之间的引用完整性
- ☒ B 统一编写, 可以提高团队合作开发的程序质量, 便于维护
- ☒ C 可以提高程序运行效率
- ☐ D 可以做到不用修改SQL语句就能适应未来需求的变化

提交

关于DBMS核心的组成结构，下面说法正确的是：

- ☒ A 一个DBMS可以提供多种不同的访问接口
- ☒ B 访问管理层向上提供的API称为访问原语
- ☐ C Parser模块可以不要
- ☒ D 单就一条SQL语句的执行来说，并发控制和恢复模块可以不要

提交

DBMS的进程结构，一般包括以下几种：

A

单线程结构

B

单进程结构

C

多线程结构

D

多进程结构

提交

关系型数据库系统的底层文件组织结构，常用的包括：

A

倒排文件

B

堆文件

C

簇集文件

D

R树索引

提交

关于B+树索引，下述说法正确的是：

A

匹配元组的地址信息存放在中间结点中

B

它是一种平衡树

C

叶结点按索引值从小到大是排好序的

D

叶结点之间由双向链表连接

提交

关于查询优化，下述说法正确的是：

A

优化用户写的SQL语句

B

由代数优化和操作优化两个步骤组成

C

确定最优的执行策略，用最少的I/O次数找出满足条件的元组

D

优化表的文件存储结构

提交

代数优化解决的问题是：

- ☐ A 将查询转化成代数运算
- ☒ B 将用户所写的查询表达式转换成结果等价、执行效率更高的形式
- ☐ C 用关系代数表达查询
- ☐ D 减少查询表达式中的除法操作

提交

操作优化解决的问题是：

- ☐ A 优化操作系统API
- ☐ B 优化文件系统，提高文件处理效率
- ☐ C 设法减少连接操作
- ☒ D 根据数据的存储结构和索引情况，找出I/O次数最少的访问路径

提交

关于查询树，下述说法正确的是：

- ☒ A 树根和每个中间结点代表一个一元/二元操作
- ☐ B 叶结点包含所需数据的地址信息
- ☐ C 叶结点是从左到右排序的
- ☒ D 从树叶到树根的顺序就是查询的执行顺序

提交

关于代数优化中的等价变换规则，下属说法正确的是：

- ☐ A 依据这些规则可以自动改写用户提交的SQL语句
- ☒ B 它相当于因式分解中的平方和公式等这类公式
- ☒ C 它们决定了对查询树可以做的变换
- ☐ D 它们决定了WHERE字句中可以使用的表达式文法

提交

if F in the form of $F1 \wedge F2$, and there are only $E1$'s attributes in $F1$, while $F2$ includes the attributes both in $E1$ and $E2$, then

$$\sigma_F(E1 \times E2) \equiv \sigma_{F2}(\sigma_{F1}(E1) \times E2)$$

按照上述规则，可以对查询树做下述变换：

- ☐ A 可以将 σ_F 拆分成 σ_{F1} 和 σ_{F2} 两个结点
- ☐ B 可以将 σ_F 拆分成 σ_{F1} 和 σ_{F2} 两个结点，并且全部压到 $\times$ 操作下面
- ☒ C 可以将 σ_F 拆分，并且将 σ_{F1} 压到 $\times$ 操作下面对 $E1$ 做选择
- ☐ D 可以将 σ_F 拆分，并且将 σ_{F2} 压到 $\times$ 操作下面对 $E2$ 做选择

提交

```
SELECT S.sname
FROM   Sailors S
WHERE  EXISTS (SELECT *
                FROM   Reserves R
                WHERE  R.bid=103 AND S.sid=R.sid)
```

对上述语句，如果所用DBMS的代数优化只支持PPT中示例的10条等价变换规则，则执行方式将是：

- ☐ A 系统报语法错误
- ☒ B 无法变换，系统直接按语句本身的两重循环逻辑进行计算
- ☐ C 系统将其拆解成两条SQL语句分别计算
- ☐ D 系统报语义错误

提交

✓ Push down the unary operations as low as possible

这是代数优化时的基本原则，这样做的目的是：

- ☐ A 精简查询树，提高遍历速度
- ☐ B 因为一元操作简单，所以要先做
- ☒ C 尽早减少连接等二元操作的操作数规模，提高查询效率
- ☐ D 因为必须先做一元操作然后才能执行二元操作

提交

在实现选择操作的计算时，如果在查询属性上有索引，则：

- ☐ A 优先使用索引，肯定能提高效率
- ☒ B 如果匹配元组过多，可能反而不如顺序扫描
- ☐ C 索引和顺序扫描同时做，看哪个快
- ☒ D 需要进行代价估算，判断使用索引是否合算

提交

关于实现连接操作的嵌套循环算法，下述说法正确的是：

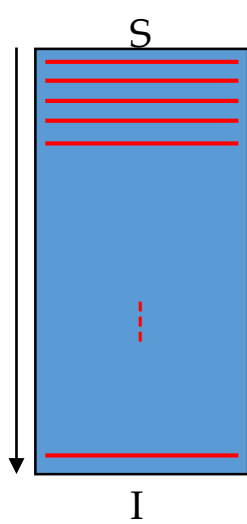
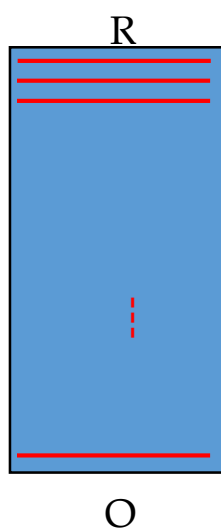
- ☒ A 它相当于两重循环，一个关系做外循环，另一个作为内循环
- ☒ B 它可以确保两个关系的所有元组两两比较，不会遗漏
- ☐ C 可以通过使用内存缓冲区，改进算法逻辑
- ☒ D 可以通过使用内存缓冲区，在算法逻辑没有改进的情况下减少磁盘I/O

提交

在用嵌套循环算法实现两个关系的连接操作时，可以通过使用内存缓冲区减少I/O次数，提高效率。如果有更多的缓冲区，为什么都给了外循环关系，而内循环关系始终只给一个缓冲区？

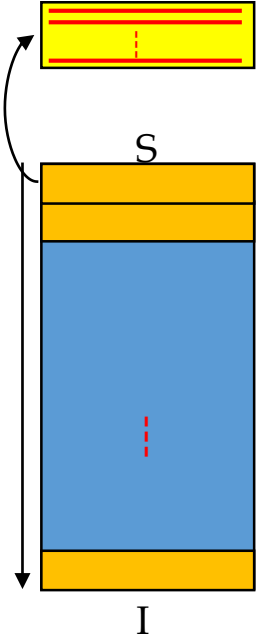
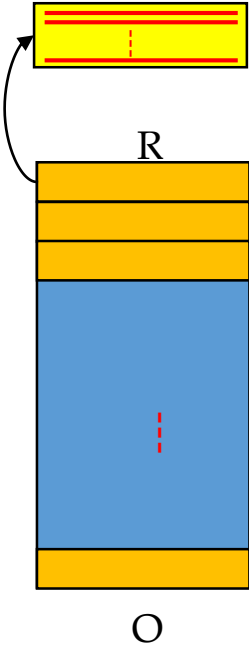
正常使用主观题需2.0以上版本雨课堂

作答



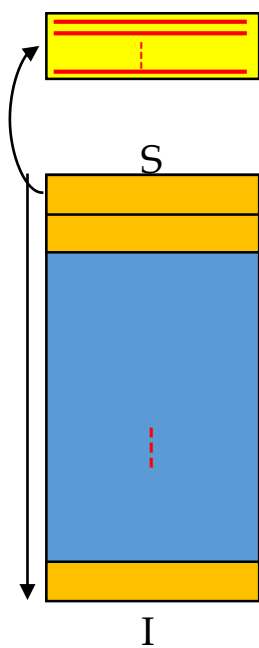
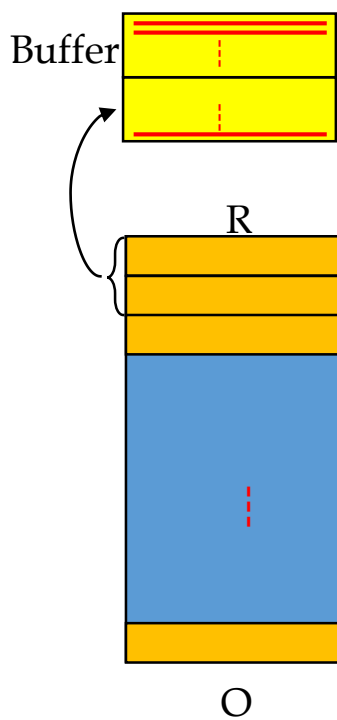
$m \times n$ 次 I/O

Buffer

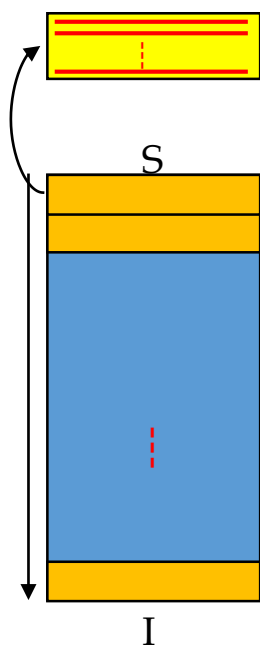
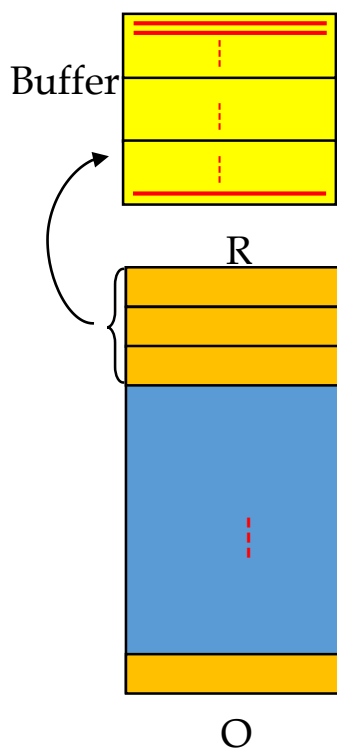


1024byte/Block
100byte/tuple

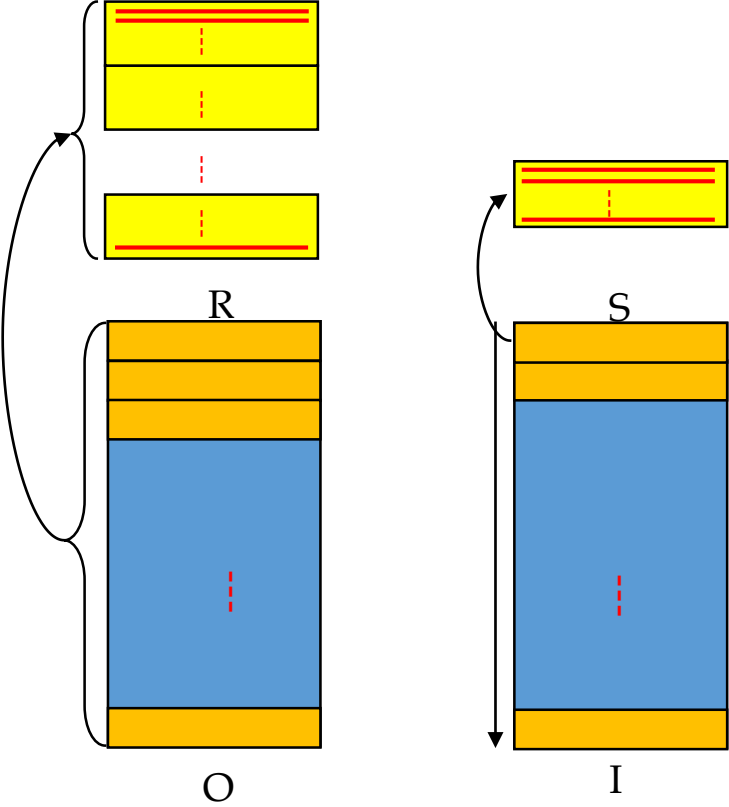
$m/10 \times n/10$
次I/O



$(m/10 \times n/10)/2$
次I/O



$(m/10 \times n/10)/3$
次I/O



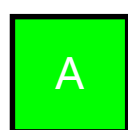
$$b_R + b_S \text{ 次 I/O}$$

关于归并扫描算法，下述说法正确的是：

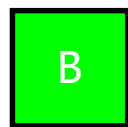
- ☒ A 参与连接的关系按照连接属性的值从小到大事先做好外排序
- ☐ B 参与连接的关系按照任意属性的值事先做好外排序
- ☒ C 对两个关系分别只需要扫描一次
- ☐ D 对内循环关系需要扫描多次

提交

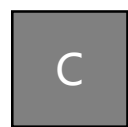
在使用索引或者Hash寻找匹配元组的连接算法中，下述说法正确的是：



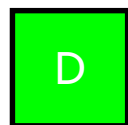
基本的算法框架仍然是嵌套循环



通过索引或者Hash寻找匹配元组，避免对内循环关系全表扫描



通过索引或者Hash寻找外循环关系中的有用元组



连接属性上有簇集索引时效果更好

提交

关于Hash Join算法，下述说法正确的是：

- ☐ A 将两个关系分别散列到一个散列文件中
- ☒ B 将两个关系散列到同一个散列文件中
- ☒ C 使用同一个Hash对两个关系的连接属性做散列
- ☐ D 对两个关系的连接属性分别定义不同的Hash函数

提交

关于几种连接实现算法，下述说法不正确的是：

- ☒ A 当除了堆文件无其它存取路径可用时，应采用归并扫描算法
- ☐ B 归并扫描算法适合于数据更新很少的应用场景
- ☐ C Hash Join需要付出事先创建散列文件的代价
- ☐ D 嵌套循环是最基本的、至少可用的连接算法

提交

关于周期性转储的恢复方法，
下列说法正确的是：

- ☒ A 它可以在遇到任何故障时将数据库恢复到备份时的一致状态
- ☐ B 它可以满足企业级应用场景的各种需求
- ☒ C 它存在丢失更新问题
- ☒ D 它只适用于简单的、小型的DBMS系统

提交

关于增量转储的备份及恢复方法，下面说法正确的是：

A

它可以解决丢失更新问题

B

它仍然属于周期性转储的方法

C

它仍然存在丢失更新问题

D

它的效果要好于一般的周期性转储方法

提交

关于基于日志的恢复方法，下述说法正确的是：

- ☒ A 日志记录了最近备份以来用户对数据库的所有改变
- ☒ B 对于数据库的每一个改变，日志中需要记录前像和后像
- ☒ C 前像指数据对象被修改前的值
- ☒ D 利用日志可以将数据库恢复到距故障最近的一致状态

提交

关于事务，下述说法正确的是：

- ☒ A 事务是用户访问数据库、以及DBMS调度管理资源的基本单位
- ☒ B 事务必须遵守ACID准则
- ☐ C 用户必须先创建事务，否则无法访问数据库
- ☒ D 用户如果没有显式创建事务，系统缺省将每条SQL语句作为一个事务

提交

在转账事务的例子中，是否可以
将if A<0 then Display "insufficient fund"
Rollback

这里的Rollback改成Commit?

☐ A 不可以，因为语法上不允许

☒ B 可以，不影响程序的执行

☐ C 可以，不影响程序的结果

☒ D 可以，但这个程序有逻辑
错误

提交

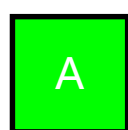
关于undo和redo操作的幂等性，下述说法正确的是：

- ☒ A 用一个数据对象的BI覆盖它实现undo，做一次和多次效果一样
- ☐ B 用多个数据对象的BI连续做undo，和对一个数据对象做undo一样
- ☒ C 用一个数据对象的AI覆盖它实现redo，做一次和多次效果一样
- ☐ D 用多个数据对象的AI连续做redo，和对一个数据对象做redo一样

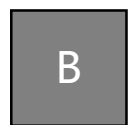
提交

关于更新策略

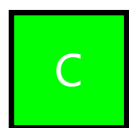
A.I→DB before commit,
下述说法正确的是：



事务执行时会直接修改数据库



事务执行时不会直接修改数据库



故障恢复时需要根据情况做undo



故障恢复时需要根据情况做redo

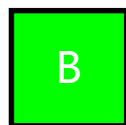
提交

关于更新策略

A.I→DB after commit ,
下述说法正确的是：



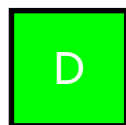
事务执行时会直接修改数据库



事务执行时不会直接修改数据库



在commit阶段没什么工作需要处理



故障恢复时需要根据情况做redo

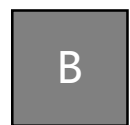
提交

关于更新策略

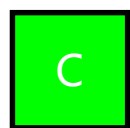
A.I→DB concurrently with commit ,
下述说法正确的是:



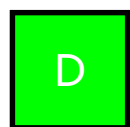
事务执行时可能会部分修改数据库



事务执行时不会直接修改数据库



故障恢复时需要根据情况做undo



故障恢复时需要根据情况做redo

提交

关于检查点（Check Point），
下述说法正确的是：

A

检查用户访问数据库的权限

B

定时设置时间点，检查事务的执行情况，完成必要的redo/undo操作

C

可以提高故障恢复时的处理效率

D

检查事务中的SQL语句是否存在错误

提交

关于并发，下述说法正确的是：

- ☐ A 只有访问数据库中不同部分数据的事务才能并发执行
- ☒ B 就是允许多个事务同时访问同一个数据库
- ☒ C 事务不能随意并发，否则可能会出问题
- ☐ D 短事务才能并发执行

提交

所谓并发时的“丢失更新”问题，指的是：

- ☐ A SQL语句出错
- ☐ B 由于系统崩溃导致数据丢失
- ☐ C 程序逻辑错误
- ☒ D 并发事务存在W-W冲突，导致执行结果错误

提交

所谓“读脏数据”，指的是：

- ☐ A 数据库状态不对
- ☐ B SQL语句错误导致结果不对
- ☒ C 并发事务间存在W-R冲突，读取了另一事务更新中的数据
- ☐ D 并发事务间存在W-W冲突

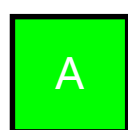
提交

所谓“不可重复的读”，指的是：

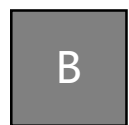
- ☐ A 同一条SELECT语句不能连续执行两次，没有意义
- ☒ B 并发事务间存在R-W冲突，事务隔离性被破坏
- ☐ C SQL语句出错导致结果不对
- ☐ D 并发事务之间存在W-W冲突

提交

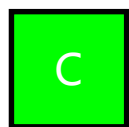
关于可串行化（Serialization），下述说法正确的是：



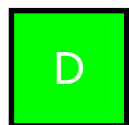
它是判断并发执行的事务结果是否正确的准则



它要求事务必须一个一个地串行执行



并发执行的事务对数据库产生的最终结果相当于它们某个串行执行序列的结果



n 个并发执行的事务可接受的正确结果有 $n!$ 个

提交

关于封锁法并发控制，下述说法正确的是：

- ☒ A 设法利用锁保证并发事务的可串行化
- ☒ B 它是最基本、最常用的一种并发控制方法
- ☒ C 它强迫有冲突的事务按照抢到锁的先后次序执行
- ☐ D 它只能使用X锁（排它锁）实现并发控制

提交

关于相容矩阵，下述说法正确的是：

A

一般用行表示数据对象上已有的锁，列表示要申请的锁

B

Y表示相容，事务必须等待

C

N表示不相容，事务必须等待

D

NL表示数据对象上没有锁

提交

Lock A

Lock B

Read (A)

Write (B)

Unlock A

Unlock B

Lock C

Read (C)

Unlock C

关于上述事务，下面说法正确的是：

A

它既不是Well-formed，也不是2PL的

B

它是Well-formed，但不是2PL的

C

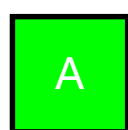
它是Well-formed，也是2PL的

D

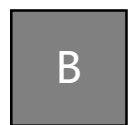
它符合Well-formed transaction的定义

提交

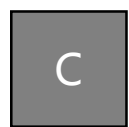
关于well-formed and two phase transactions的定理，下述说法正确的是：



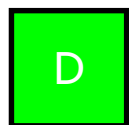
它是封锁法正确性的理论保证



它来源于实践，没有理论证明



只要符合定理要求，DBMS就不再会有并发方面任何问题



它可以保证并发事务的可串行化

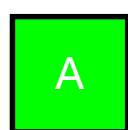
提交

所谓恢复时的多米诺现象，指的是：

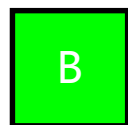
- ☐ A 多个事务连续执行导致的负载问题
- ☐ B 由于系统故障导致的多个事务滚回
- ☐ C 由于事务之间竞争资源导致互相干扰
- ☒ D 由于一个事务读了另一事务尚未提交的数据，那个事务滚回时这个事务也要跟着滚回，导致连锁滚回

提交

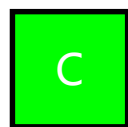
关于 (S, X) 锁协议, 下面说法正确的是:



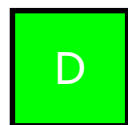
引入了共享锁, 使得并发事务可以同时读同一数据



相比X锁协议, 性能得到提高



因为多了一种锁, 系统开销有所上升



事务在读某个数据时, 如果用X锁替代S锁也不会导致错误

提交

关于 (S, U, X) 锁协议, 下述说法正确的是:

A

U锁的排它性比S锁强

B

它和A.I $\rightarrow$ DB after commit 配合, 可得到更好效果

C

它和A.I $\rightarrow$ DB before commit 配合, 可得到更好效果

D

两个更新事务可同时获得所修改数据对象的U锁

提交

关于死锁和活锁，下述说法正确的是：

A

死锁和活锁只是说法不同，其实没有差别

B

活锁比死锁容易解决

C

它们和操作系统中的死锁和活锁本质是一样的

D

死锁可以不必理会，由用户自行解决

提交

关于死锁检测方法，下述说法正确的是：

A

超时法简单好用，比等待图方法用途更广

B

等待图法是正规的方法，多用在企业级系统中

C

等待图法动态构建等待图，如果出现环路意味发生死锁

D

当发现死锁时需要选择一个牺牲者，打破循环等待链

提交

关于Transaction Retry的死锁避免方法， 下属说法正确的是：

A

引入时间戳概念，使等待图中只有年老事务等待年轻事务的单方向等待，避免出现环路

B

引入时间戳概念，使等待图中只有年轻事务等待年老事务的单方向等待，避免出现环路

C

它虽然能够避免死锁，但可能导致活锁

D

事务retry的时候更换当前时间作为新时间戳也不影响效果

提交

关于死锁避免方法，为什么说把操作系统中可用的Requesting all locks at initial time of transaction方法用于数据库系统，理论上可行实际却做不到？

正常使用主观题需2.0以上版本雨课堂

作答

关于数据库一致性被破坏的问题，下述说法正确的是：

- ☒ A 由于系统故障导致的破坏，是由恢复机制负责解决的
- ☒ B 由于并发冲突导致的破坏，是由并发控制机制负责解决的
- ☒ C 人为导致的故意或无意破坏，属于数据库安全问题
- ☒ D 输入错误数据导致的破坏，属于完整性约束问题

提交

关于数据库系统的审计追踪功能，下述说法正确的是：

- ☐ A 审查登录用户的身份
- ☐ B 审查用户提交的SQL语句
- ☒ C 记录登录用户的身份以及执行的相关操作，以便发生问题时追查责任
- ☐ D 审查用户的权限，避免非法操作

提交

所谓统计数据库的安全，指的是：

- ☐ A 要加强用户权限管理，避免非法统计计算
- ☐ B 要对数据进行加密，防止泄密
- ☐ C 要加强用户密码控制，防止系统被黑客攻破
- ☒ D 要在允许对数据进行公开统计查询基础上，避免个体隐私数据被推算出来

提交

关于针对统计数据库的个体追踪器，下属说法正确的是：

- ☒ A 它其实就是能够唯一确定某个特定个体的布尔表达式
- ☒ B 理论上针对任何个体的追踪器总是能找到的
- ☐ C 理论上统计数据库是可以做到绝对安全的
- ☒ D 这有点像加密和破译之间的关系，魔高一尺、道高一丈，关键在代价

提交

关于完整性约束，下述说法正确的是：

A

它规定了SQL语句的格式

B

它描述了数据库中合法数据应该满足的条件

C

它可以防止非法用户对数据库的破坏

D

它可以在一定程度上保证数据的一致性

提交

关于引用完整性约束，下述说法正确的是：

A

它是一种显式约束

B

它是包含在数据库模式定义中的隐式约束

C

它规定外键的值在所引用的表的相应属性中必须存在

D

它和插删改操作密切相关

提交

如果 K_1 是关系 r_1 的主键， α 是关系 r_2 中的外键、它引用了关系 r_1 的 K_1 ，则关于级联删除，下述说法正确的是：

A

当 r_1 删除一条元组 t_1 时， r_2 中所有满足 $t_2[\alpha]=t_1[K_1]$ 的元组 t_2 都要随之删除

B

当 r_2 删除一条元组 t_2 时， r_1 中满足 $t_1[K_1]=t_2[\alpha]$ 的元组 t_1 也要随之删除

C

级联删除是保证引用完整性的解决方法之一

D

当 r_1 删除一条元组 t_1 时， r_2 中所有的元组都要随之删除

提交

如果K1是关系r1的主键， α 是关系r2中的外键、它引用了关系r1的K1，则在对r1、r2做更新时，下述说法正确的是：

A

当更新r1中元组t1的任意属性值时，都要考虑级联更新

B

当更新r2中元组t2的 α 值时，情形类似于在r2中插入新元组

C

当更新r1中元组t1的K1值时，情形类似于在r1中删除元组，需考虑级联更新

D

当更新r2中元组t2的任意属性值时，都需要检查引用完整性

提交

```
CREATE TABLE Reserves
( sname CHAR(10),
  bid INTEGER,
  day DATE,
  PRIMARY KEY (bid, day),
  CONSTRAINT noInterlakeRes
  CHECK ('Interlake' <>
        ( SELECT B.bname
          FROM Boats B
          WHERE B.bid=bid)))
```

noInterlakeRes约束的语义是：

- ☐ A 任何一条船的名字都不能叫Interlake
- ☒ B 名字叫Interlake的船不能外借
- ☐ C 水手只能租借名字叫Interlake的船
- ☐ D 名字叫Interlake的船只能外借一次

提交

单选题 2分



create table Teachers(
...

Age int **check**(age>18 and age<30),
...

)

create table Teachers(
...

Age int ,
...

constraint ck_age **check** (age > 18 and age <30)

)

对于上述两个约束定义，下述说法正确的是：

- ☐ A 这是两个完全不同的约束
- ☐ B 第一个约束语法不对
- ☐ C 第二个约束语法不对
- ☒ D 两个约束作用相同，只不过一个在列级定义，一个在表级定义

提交

```
alter table Users  
add constraint CK_Balance check (Balance > 0)
```

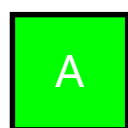
上述SQL语句的作用是：

- ☒ A 为Users表增加一条约束CK_Balance，保证余额大于0
- ☐ B 将Users表改名为CK_Balance
- ☐ C 将Balance>0的表改名为Users
- ☐ D 在Users表中增加一个属性CK_Balance

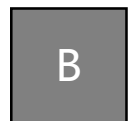
提交


```
CREATE ASSERTION smallClub  
CHECK  
( (SELECT COUNT (S.sid) FROM Sailors S)  
+(SELECT COUNT (B.bid) FROM Boats B) < 100 )
```

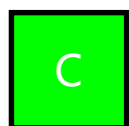
对上述SQL语句，下面说法正确的是：



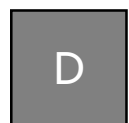
它定义了一个涉及两个表的显式约束



它要求两个表必须同时更新



断言不依赖于特定表，可以更方便地定义显式约束



它要求水手的人数不能超过100

提交

关于触发器，下述说法正确的是：

A

它是一个门电路，通电会自动触发动作

B

它是一种当数据库发生特定变化时，可以自动执行的过程

C

它也被称为ECA规则

D

它可以在一定程度上实现主动数据库的功能

提交

```
CREATE TRIGGER 不能再订绿船
BEFORE INSERT ON Reserves
REFERENCING NEW AS N
FOR EACH ROW
WHEN (N.bid IN (SELECT bid FROM Boats WHERE
                color = 'Green') AND
      N.sid IN (SELECT sid FROM Reserves, Boats
                WHERE Reserves.bid=Boats.bid AND color='Red'))
ROLLBACK;
```

对于上述触发器，下面说法正确的是：

- ☒ **A** 它的事件 (E) 是对Reserves表做INSERT之前
- ☒ **B** 它引用的是INSERT操作的新值
- ☐ **C** 它的条件 (C) 是检查新预定的船是否为绿船
- ☒ **D** 它的动作 (A) 是滚回，拒绝此次插入操作

提交

```
CREATE TRIGGER 不能再订绿船
BEFORE INSERT ON Reserves
REFERENCING NEW AS N
FOR EACH ROW
WHEN (N.bid IN (SELECT bid FROM Boats WHERE
                color = 'Green') AND
      N.sid IN (SELECT sid FROM Reserves, Boats
                WHERE Reserves.bid=Boats.bid AND color='Red'))
ROLLBACK;
```

对于上述触发器，它的约束的语义是：

- ☐ A 水手不能预定绿船
- ☐ B 水手不能预定红船
- ☒ C 如果一个水手已经预定过红船，则他不能再预定绿船
- ☐ D 水手必须先预定红船，然后才能预定绿船

提交

关于触发器的级联触发问题，下面说法正确的是：

A

它是由与引用完整性相关的级联更新造成的

B

它是指一个触发器触发了多个动作 (A)

C

由于一个触发器的动作导致另一个触发器的事件，而且条件满足，从而连续触发多个触发器

D

触发器的定义必须慎重，团队开发时不能任由每个成员随意定义触发器

提交

关于触发器在DBMS系统中的实现方式，下述说法正确的是：

A

可以采用松耦合方式，在已有DBMS核心之上包一层，在一定程度上扩展对触发器的支持

B

可以采用紧耦合的方法，从DBMS设计之初就考虑对触发器的支持

C

可以采用嵌入的方法，适当修改DBMS的事务执行模块，扩展一些类似触发器的功能

D

松耦合和嵌入的方法都属于打补丁的策略，不能实现完整的触发器功能

提交

关于数据依赖，下属说法正确的是：

A

数据依赖是指关系的不同属性之间存在的某种依赖关系

B

函数依赖是一种数据依赖

C

多值依赖是一种函数依赖

D

连接依赖就是表之间的连接关系

提交

关于1NF，下述说法正确的是：

A

它是关系数据模型固有的约束

B

它是指一张表初次定义时的模式

C

它是指任何属性只能是基本数据类型

D

俗话说就是不允许表中套表

提交

关于2NF，下述说法正确的是：

A

属性间存在连接依赖

B

它也同时是1NF

C

属性间存在部分函数依赖

D

消除了属性对主键的部分函数依赖

提交

关于3NF，下述说法正确的是：

A

属性间存在连接依赖

B

它也同时是2NF

C

在2NF基础上消除了属性间的传递函数依赖

D

只需消除属性对主键的部分函数依赖

提交

关于插入异常，下述说法正确的是：

- ☐ A 由于SQL语句错误，导致无法插入数据
- ☐ B 由于完整性约束，导致想插入的数据被系统拒绝
- ☒ C 由于模式定义不合理，导致所需数据无法顺利插入
- ☐ D 由于权限设置不合理，导致所需数据无法插入

提交

关于删除异常，下述说法正确的是：

- ☐ A 由于SQL语句错误，导致有用数据被误删
- ☒ B 由于模式定义不合理，导致有用数据被连带删除
- ☐ C 由于完整性约束，导致想删除的数据无法删除
- ☐ D 由于权限设置不合理，导致数据无法删除

提交

关于更新异常（或者说更新困难），下述说法正确的是：

- ☒ A 由于模式定义不合理导致大量数据冗余，更新时难以保证一致性
- ☐ B 由于完整性约束，导致数据更新操作被系统拒绝
- ☐ C 由于模式定义不合理，导致所需更新操作无法执行
- ☐ D 由于权限设置不合理，导致所需数据无法更新

提交

关于“一事一地”的原则，下述说法正确的是：

- ☐ A 对于现实世界中的一个实体集，只能创建一张表
- ☒ B 即一张表只管一件事情，使表模式符合3NF
- ☒ C 如果表模式不符合一事一地原则，可通过模式分解将其分解成多张表
- ☐ D 一个库里面同时只能创建一张表

提交

关于ER模型和ER图，下列说法正确的是：

- ☒ A 它可以在概念层面描述企业管理所涉及的数据及其联系
- ☒ B 它不涉及具体实现细节，更加便于和用户交流讨论
- ☒ C 它与所使用的具体DBMS产品无关
- ☐ D 基于ER图可以直接创建数据库

提交

关于数据库设计方法，下述说法正确的是：

A

所谓数据库设计只是建表而已，无所谓方法

B

面向数据的方法深入分析数据项的本质及其内在关系，适合复杂数据库的设计

C

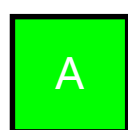
面向过程的方法以业务流程的实现为核心，可以快速实现信息系统的开发

D

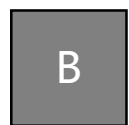
面向过程的方法忽视了对数据内在关系的分析，难以适应需求的变化

提交

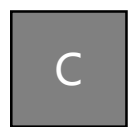
关于面向过程的数据库设计方法的优缺点，下述说法正确的是：



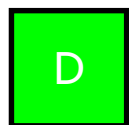
优点是开发进度快，可以较快实现所需业务功能



优点是所开发出的信息系统性能高



缺点是所开发出的信息系统性能差



缺点是存在数据冗余、冲突的隐患，不能很好适应需求的变化

提交

关于面向数据的数据库设计方法的优缺点，下述说法正确的是：

A

优点是开发进度快，可以较快实现所需业务功能

B

优点是消除了数据在结构上的冗余和冲突，所开发出的系统质量有保证

C

缺点是需求分析和概念设计阶段工作量较大，前期进度慢

D

缺点是所开发出的信息系统性能差

提交

关于数据库设计流程，下述说法正确的是：

A

需求分析的关键在于正确理解用户的业务，以及这些业务所涉及的数据及处理

B

逻辑设计阶段需要考虑所用DBMS的技术特性

C

所谓外模式主要由基表组成

D

所谓内模式就是数据库中表的底层存储格式及索引等

提交

关于数据库设计与软件工程之间的关系，下述说法正确的是：

- ☒ A 数据库设计流程和软件工程的总体流程是吻合的
- ☐ B 数据库设计是独立的过程，和软件工程无关
- ☒ C 数据库设计是企业信息系统设计过程的核心工作
- ☒ D 在软件工程的需求分析阶段，重点是围绕数据处理的需求

提交

需求分析时需要向用户进行详细的调研，在企业不同业务部门调研时可能会发现某些数据存在概念冲突，所谓概念冲突（Concept conflicts）是指：

- ☐ A 同一概念在不同子系统（部门）中命名不同
- ☐ B 不同概念在不同子系统（部门）中使用了相同的名字
- ☒ C 同一概念在一个子系统（部门）中可能作为实体，而在另一个子系统中可能被当作属性
- ☐ D 同一概念在不同子系统（部门）中数据类型和值域不同

提交

关于概念设计阶段的工作，下述说法正确的是：

- ☒ A 基于需求分析结果，识别出实体集和实体集间的联系
- ☒ B 一般采用ER图或者UML图的方式表达概念模式
- ☐ C 要依据所用DBMS的特性，决定各属性的数据类型
- ☒ D 最好要有ERWin、Rose这样的设计工具支持

提交

关于逻辑设计阶段的工作，下述说法正确的是：

A

完成索引的定义

B

根据ER图表达的概念模式，确定目标DBMS中表和视图的定义

C

遵循的基本标准是3NF或者BCNF范式

D

需要根据所用DBMS特性，确定每个属性的具体数据类型

提交

在逻辑设计阶段，为什么要考虑适当的“逆规范化”？为什么不能机械地追求3NF？

正常使用主观题需2.0以上版本雨课堂

作答

关于物理设计阶段的工作，下述说法正确的是：

- ☒ A 此时重点考虑的是效率，决定数据的物理存取路径
- ☒ B 需要考虑所用DBMS、操作系统、甚至服务器硬件的特性
- ☒ C 需要考虑每个表的文件存储结构和索引的设计
- ☒ D 需要考虑数据分区设计

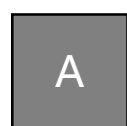
提交

关于数据库分区，下面说法正确的是：

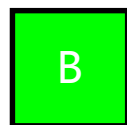
- ☒ A 是一种物理数据库设计技术，将过大的表分割成若干分区
- ☒ B 可以按照元组做水平分区或者按照属性做垂直分区
- ☒ C 通过分区可减少特定SQL操作中数据读写的总量以提高速度
- ☐ D 可以根据用户的权限划分数据

提交

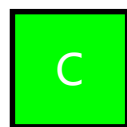
关于分布式数据库技术出现的技术背景，下面说法正确的是：



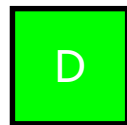
因为一个企业的部门是分布的，所以数据库要分布



当时基于局域网的分布式计算机系统刚刚出现



当时单台计算机处理能力很弱，而大型计算机很贵



希望由多台连网的计算机互相合作，共同完成企业级数据库的管理

提交

关于分布式数据库物理上分布、逻辑上集中的特点，下属说法正确的是：

- ☒ A 一个数据库中的数据物理上分散存储在连网的多台计算机上
- ☐ B 用户看到的是多个不同的数据库
- ☒ C 用户看到的是一个数据库，感觉不到数据的分布
- ☐ D 数据集中存放在网络中配置最高的计算机上

提交

分布式数据库系统可用性好的优点，指的是：

- ☐ A 分布式数据库系统比集中式数据库系统好用
- ☐ B 分布式数据库系统比集中式数据库系统简单
- ☒ C 由于数据在多个结点有多个复本，单个节点的故障不会导致整个系统不可用
- ☐ D 所有结点崩溃了系统仍然可用

提交

关于分布式数据系统与集中式数据库系统的差别，下述说法正确的是：

A

查询优化的优化目标不同

B

并发控制需要考虑整个网络中的全局冲突，更复杂

C

恢复机制需要考虑不同结点故障的组合，更复杂

D

数据分布的策略比集中式数据库更难

提交

关于数据分割，下述说法正确的是：

- ☐ A 水平分割是按照属性将关系分成若干裂片
- ☒ B 水平分割是按照元组将关系分割成若干裂片
- ☐ C 垂直分割是先按属性分割关系再按元组做进一步分割
- ☒ D 垂直分割是按照属性将关系分割成若干裂片

提交

下面哪个不是由于数据分布带来的问题？

- ☐ A 多副本的一致性问题
- ☐ B 分布一致性问题
- ☐ C 全局查询需要翻译（转换）成裂片查询
- ☒ D 数据分区设计

提交

关于数据分割方案以及裂片分布方案的设计，下述说法正确的是：

A

这是分布式数据库设计时需要解决的主要问题

B

通过数据分割在满足结点数据需求的同时减少数据冗余，提高系统效率

C

一般来说数据分割的越细越好

D

数据分割的依据是各结点的数据访问需求

提交

关于联邦式数据库系统，下述说法正确的是：

A

它和分布式数据库技术毫无关系

B

它也是一种分布式数据管理技术

C

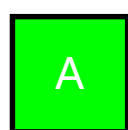
整个系统没有统一的全局模式，各结点用户看到的数据库都是不一样的

D

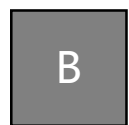
整个系统有统一的全局模式，用户感觉不到数据的分布

提交

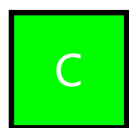
关于联邦式数据库的联邦模式，下述说法正确的是：



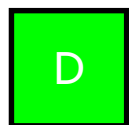
它是每个联邦结点上的用户所能看到的所有数据



它是由全局模式平均划分的结果



它由每个结点的本地模式和通过协商而得的输入模式组成



用户针对联邦模式写查询，需要转换成对本地数据库和与输入模式相对应的远程数据库的查询

提交

关于分布式数据库系统的查询优化，下面说法正确的是：

- ☒ A 它的原理和基本目的和集中式数据库系统是一样的
- ☒ B 在代数优化阶段和集中式数据库系统几乎没有差别
- ☒ C 在操作优化阶段考虑的优化目标是减少网上数据传输
- ☒ D 产生的结果是全局执行计划

提交

关于分布式事务，下述说法正确的是：

- ☐ A 分布式数据库系统中的事务都是分布式事务
- ☒ B 它是由运行在不同结点的多个子事务组成的事务
- ☒ C 组成一个分布式事务的多个子事务必须同时提交或滚回
- ☒ D 分布式事务的提交需要特殊协议的支持（2PC）

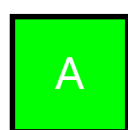
提交

关于分布式数据库系统中的并发控制，下述说法正确的是：

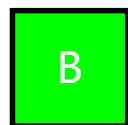
- ☒ A 基本原理没有变，控制的目标仍然是可串行化
- ☒ B 仍然可以使用封锁法
- ☒ C 新问题是多复本带来的全局冲突和通讯开销问题
- ☐ D 不需要考虑死锁问题了

提交

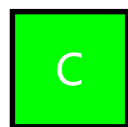
关于数据仓库的定义，下面说法正确的是：



数据仓库就是一种特殊的数据库



数据仓库的数据面向决策主题，支持决策分析



数据仓库是一种以读为主的数据库



数据仓库是与现有数据库技术完全不同的全新技术

提交

关于ETL，下述说法正确的是：

A

它是数据仓库的建模工具

B

它是数据仓库建设过程中的重要一环

C

它负责从数据源抽取决策所需数据，转换成分析所需的格式，并加载到数据仓库中

D

有很多第三方工具软件专门支持ETL工作

提交

关于OLTP和OLAP，下述说法正确的是：

A

OLTP是对数据的分析型应用，辅助决策支持

B

OLAP是对数据的分析型应用，辅助决策支持

C

OLAP是对数据的操作型应用，日常事务处理

D

OLTP是对数据的操作型应用，日常事务处理

提交

关于MOLAP和ROLAP，下述说法正确的是：

A

将多维数据立方体在物理上组织并存储成多维数组形式的系统，称为MOLAP

B

将多维数据立方体以关系数据库的表的形式存储的系统，称为ROLAP

C

ROLAP就是用关系数据库开发信息系统

D

MOLAP就是用多维数组组织数据开发信息系统

提交

关于星型模式，下述说法正确的是：

- ☒ A 将多维数据立方体用事实表加若干维表的形式表达
- ☒ B 事实表反映了度量值和不同分析因素（维）之间的关系
- ☒ C 维表表达了某个分析因素（维）的详细信息
- ☒ D 维表不分解，可能不满足3NF

提交

关于雪花模式，下述说法正确的是：

A

它和星型模式本质相似

B

将星型模式的维表做进一步分解，使其满足3NF

C

比星型模式的统计查询效率更高

D

事实表和维表之间是外键引用关系

提交

关于实视图（Materialized View），下述说法正确的是：

A

它和视图一样是基于基表定义的映射，但其数据是物化保存在数据库中的

B

可以对实视图执行更新操作

C

一个 n 维星型模式可以定义 2^n 个实视图，将其各种维度组合的统计值都算好

D

利用实视图可大幅提高统计计算效率，但只适用于数据仓库

提交

下述哪个不是基于数据立方体的OLAP查询操作：

☐ A 上卷与下钻

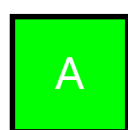
☐ B 旋转

☐ C 切片

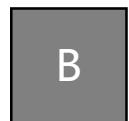
☒ D 外连接

提交

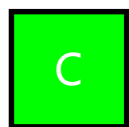
关于CUBE操作和CUBE子句，
下述说法正确的是：



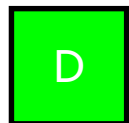
它是对原有SQL文法和功能的扩充



它可以取代原有的SQL语句



一个CUBE操作可以替代
多条SELECT语句



可以在原有SELECT语句中扩充CUBE
子句实现多维统计计算

提交

关于位图索引，下述说法正确的是：

A

它仍然是一种索引

B

它最大特点是索引本身就是数据

C

很多计算通过位图索引可以直接获得结果，不必再去访问数据本身

D

它也可以广泛用于OLTP数据库

提交

关于数据挖掘的定义，下述说法正确的是：

- ☒ A 它的目的是从海量数据中挖掘出可能潜藏的规则
- ☒ B 这些规则是人们以前不知道或者没注意的
- ☒ C 这些规则应该是确定的
- ☒ D 这些规则应该有助于我们改进经营决策

提交

数据挖掘工作要想取得成功，
下述哪些是必备的支撑条件：

A

需要大量人力进行人海战术

B

需要有海量的相关商业数据集

C

需要有强大的高性能计算机或云计算平台

D

需要有高效的数据挖掘算法

提交

决策树算法解决的是下面哪类问题：

☒ A 分类问题（预计）

☐ B 聚类问题

☐ C 关联分析

☐ D 预测问题

提交

著名的啤酒与尿片的例子，它属于下面哪种数据挖掘问题：

☐ A 分类问题

☐ B 聚类问题

☒ C 关联分析

☐ D 预测问题

提交

评价所发现的关联规则有效性的主要指标是下述哪些：

A

正确率

B

支持度

C

召回率

D

置信度

提交